CS143 Spring 2026 – Written Assignment 2

Due Monday, April 27, 2026 11:59 PM PDT

This assignment covers context free grammars and parsing. You may discuss this assignment with other students and work on the problems together. However, your write-up should be your own individual work, and you should indicate in your submission who you worked with, if applicable. Assignments can be submitted electronically through Gradescope as a PDF by 11:59 PM PDT. Please review the the course policies for more information: `https://web.stanford.edu/class/cs143/policies/`. A LaTeX template for writing your solutions is available on the course website. If you need to draw parse trees in LaTeX, you may use the `forest` package: `https://ctan.org/pkg/forest`.

1. Give a context-free grammar (CFG) for each of the following languages. Any grammar is acceptable—including ambiguous grammars—as long as it has the correct language. The start symbol should be S.

   (a) The set of all strings over the alphabet $\{2, -, +\}$ representing valid arithmetic expressions where each integer in the expression is a single digit and the expression evaluates to some value ≥ 0. Note that the empty string ε is *not* in the language.
   Example Strings in the Language:

   2+2 2-2+2 -2-2+2+2

   Strings not in the Language:

   -2 -2+22 2++2-2

   Solution:

Gramatika (startni simbol S):

$$S \to 2\,D \mid -2\,U$$
$$T \to +2\,T \mid T+2 \mid +2\,T-2\,T \mid -2\,T+2\,T \mid \varepsilon$$
$$D \to T \mid T-2\,T$$
$$U \to T+2\,T$$

Objašnjenje

Izraz mora biti sintaksno valjan i imati vrijednost ≥ 0.
Pošto svaki član daje $+2$ ili -2, ≥ 0 znači da plusa
mora biti najmanje koliko i minusa. Uloge neterminala:

- T — niz $+2/-2$ sa $\#(+2) \geq \#(-2)$.
- D — ostatak iza $+2$; trpi jedan minus viška.
- U — ostatak iza -2; traži bar jedan plus viška.
- S — bira prvi član ($2D$ ili $-2U$); zato vrijednost ≥ 0.

(b) The set of all strings over the alphabet $\{0,1\}$ with exactly one more 0 than 1's.
Example Strings in the Language:

$$0 \qquad 100 \qquad 010 \qquad 0010011$$

Strings not in the Language:

$$\varepsilon \qquad 00 \qquad 01 \qquad 0110 \qquad 0100$$

Solution:

Gramatika (startni simbol S):

$S \to A\,0\,A$

$A \to 0\,A\,1\,A \mid 1\,A\,0\,A \mid \varepsilon$

Objašnjenje

Treba nam $\#0 = \#1 + 1$, tj. tačno jedna nula viška.
Uloge neterminala:

- A — uravnoteženi string ($\#0 = \#1$). Pravila $0A1A$ i $1A0A$ uparuju svaku nulu sa po jednom jedinicom; ε je prazan.
- S — ubacuje jednu višak-nulu između dva uravnotežena dijela ($A\,0\,A$), pa ukupno ostaje tačno jedna nula viška.

(c) The set of all strings over the alphabet $\{0,1\}$ in the language $L : \{0^i 1^j 0^k \mid j = i + k\}$.
Example Strings in the Language:

ε	10	01	0110	000111

Strings not in the Language:

0	00	001	0010	01110

Solution:

Gramatika (startni simbol S):

$S \to X\,Y$

$X \to 0\,X\,1 \mid \varepsilon$

$Y \to 1\,Y\,0 \mid \varepsilon$

Objašnjenje

Jezik je $\{\, 0^i\, 1^j\, 0^k : j = i + k \,\}$ — svih j jedinica je u sredini, a treba ih tačno $i + k$. Uloge neterminala:

- X — generiše $0^i\, 1^i$: svaka vodeća nula dobija po jednu jedinicu ($0\,X\,1$). Pokriva lijevi dio i jedinica.
- Y — generiše $1^k\, 0^k$: svaka prateća nula dobija po jednu jedinicu ($1\,Y\,0$). Pokriva desni dio k jedinica.
- $S \to XY$ — spaja u $0^i\, 1^i 1^k\, 0^k = 0^i\, 1^{i+k}\, 0^k$.

(d) The set of all strings over the alphabet $\{[,], \{, \}, ,\}$ which are sets. We define a set to be a collection of zero or more comma-separated arrays enclosed in an open brace and a close brace. Similarly, we define an array to be a collection of zero or more comma-separated sets enclosed in an open bracket and a close bracket. **Note** that "," (comma) is in the alphabet.
Example Arrays:

$$[\{\}, \{\}] \qquad [] \qquad [\{[], []\}]$$

Example Sets:

$$\{[]\} \qquad \{\} \qquad \{[\{\}], []\}$$

Example Strings in the Language:

$$\{\} \qquad \{[], [\{[]\}]\} \qquad \{[\{\}, \{\}, \{\}], []\}$$

Strings not in the Language:

$$[] \qquad \{\{\}\} \qquad \{[[]]\}$$

Solution:

Gramatika (startni simbol S):

$S \to \{\, A \,\}$
$A \to \varepsilon \mid B$
$B \to R \mid R\,, B$
$R \to [\, C \,]$
$C \to \varepsilon \mid D$
$D \to S \mid S\,, D$

Objašnjenje

Skup sadrži nizove, a niz sadrži skupove — međusobna rekurzija.
Uloge neterminala:

- S — skup: vitičaste zagrade oko liste nizova, $\{A\}$.
- R — niz: uglaste zagrade oko liste skupova, $[C]$.
- A, C — liste (ε = prazno, ili neprazna lista).
- B — neprazna lista nizova; D — neprazna lista skupova (elementi razdvojeni zarezom).

2. Consider the following grammar for if-then-else expressions that involve a variable x:

$$E \to \text{if } x \text{ then } E \mid M$$
$$M \to \text{if } x \text{ then } M \text{ else } E \mid x$$

Is this grammar ambiguous or not? If yes, give an example of an expression with two different parse trees and draw the two parse trees. If not, explain why that is the case.

Solution:

Zaključak: gramatika NIJE dvosmislena.

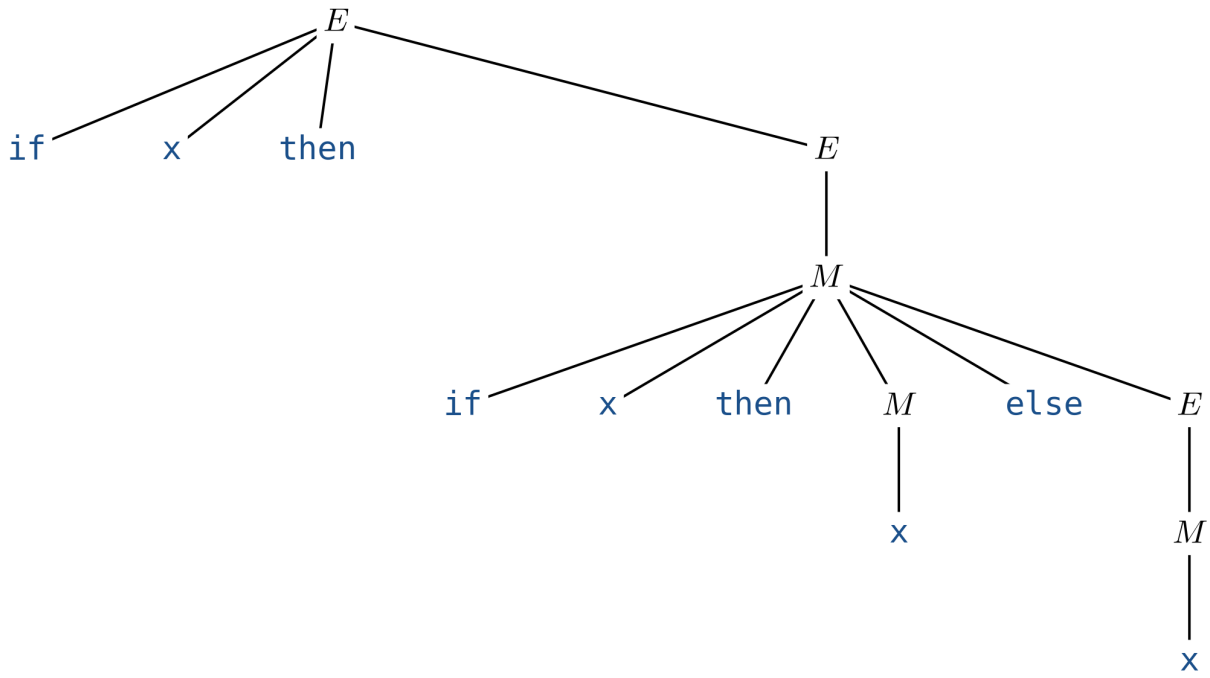
Gramatika dijeli izraze u dvije klase:

- M — „upareni“ izraz: svaki if … then ima svoj else ($M \to$ if x then M else $E \mid$ x).
- E — opšti izraz, smije imati „otvoreni“ if bez else-a ($E \to$ if x then $E \mid M$).

Ključ je u pravilu $M \to$ if x then M else E: then-grana mora biti M (uparena), pa unutar nje ne može stajati otvoreni if koji bi „preuzeo“ else. Zato se svaki else jednoznačno veže za najbliži (unutrašnji) if.

Klasičan izraz if x then if x then x else x ima samo jedno stablo: else pripada unutrašnjem if-u, a spoljašnji if je otvoren ($E \to$ if x then E).

Jedinstveno parse-stablo za if x then if x then x else x**:**



3. (a) Left factor the following grammar:

$$S \to I \mid I - J \mid I + K$$
$$I \to (J - K) \mid (J)$$
$$J \to K1 \mid K2$$
$$K \to K3 \mid \varepsilon$$

Solution:

3. (a) Lijevo faktorisanje

Polazna gramatika:

$$S \to I \mid I - J \mid I + K$$
$$I \to (J - K) \mid (J)$$
$$J \to K1 \mid K2$$
$$K \to K3 \mid \varepsilon$$

Rezultat (izvučeni zajednički prefiksi):

$$S \to I\, S'$$
$$S' \to - J \mid + K \mid \varepsilon$$
$$I \to (\, J\, I'$$
$$I' \to - K\,) \mid)$$
$$J \to K\, J'$$
$$J' \to 1 \mid 2$$
$$K \to K3 \mid \varepsilon$$

Objašnjenje

Faktorisanje izvlači najduži zajednički prefiks alternativa da predictive (LL) parser ne mora da nagađa koju granu da uzme:

- S: sve tri grane počinju sa I — izvučeno I, ostatak ide u S'.
- I: obje grane počinju sa $($ J — izvučeno $($ J, ostatak u I'.
- J: obje počinju sa K — izvučeno K, ostatak u J'.
- K: $K3$ i ε nemaju zajednički prefiks, pa ostaje (lijevo faktorisanje ne uklanja rekurziju — to je posao iz dijela b).

(b) Eliminate left recursion from the following grammar:

$$S \to STS \mid ST \mid T$$
$$T \to Ta \mid Tb \mid U$$
$$U \to T \mid c$$

Solution:

3. (b) Eliminacija lijeve rekurzije

Polazna gramatika:

$$S \to STS \mid ST \mid T$$
$$T \to Ta \mid Tb \mid U$$
$$U \to T \mid c$$

Rezultat (bez lijeve rekurzije):

$$S \to TS'$$
$$S' \to TSS' \mid TS' \mid \varepsilon$$
$$T \to cT'$$
$$T' \to aT' \mid bT' \mid \varepsilon$$

Objašnjenje

1) Ciklus $T \to U \to T$: zamijenimo U u pravilu $T \to U$ sa $U \to T \mid c$, dobijemo $T \to Ta \mid Tb \mid T \mid c$, pa uklonimo trivijalno $T \to T \Rightarrow\ T \to Ta \mid Tb \mid c$. ($U$ postaje nedostižan.)
2) Direktna LR u T ($T \to Ta \mid Tb \mid c$): $T \to cT'$, $T' \to aT' \mid bT' \mid \varepsilon$.
3) Direktna LR u S ($S \to STS \mid ST \mid T$): $S \to TS'$, $S' \to TSS' \mid TS' \mid \varepsilon$.

4. Consider the following CFG, where the set of terminals is $\{a, b, c, d, \#, ?\}$:

$$S \to \#UT \mid T?$$
$$T \to aS \mid bUc \mid \varepsilon$$
$$U \to aSc \mid bTd$$

4. (a) FIRST skupovi

FIRST(S) = { #, ?, a, b }

FIRST(T) = { a, b, ε }

FIRST(U) = { a, b }

(b) FOLLOW skupovi

FOLLOW(S) = { $, ?, c, d }

FOLLOW(T) = { $, ?, c, d }

FOLLOW(U) = { $, ?, a, b, c, d }

(c) LL(1) tabela parsiranja

	a	**b**	**c**	**d**	**#**	**?**	**$**
S	S→T?	S→T?	—	—	S→#UT	S→T?	—
T	T→aS	T→bUc	T→ε	T→ε	—	T→ε	T→ε
U	U→aSc	U→bTd	—	—	—	—	—

(d) Show the sequence of stack, input and action configurations that occur during an LL(1) parse of the string "$\#a?ca?$". At the beginning of the parse, the stack should contain a single S.

Solution: **(d) LL(1) parsiranje stringa #a?ca?**

Stek	**Ulaz**	**Akcija**
S	#a?ca?$	S→#UT
#UT	#a?ca?$	match #
UT	a?ca?$	U→aSc
aScT	a?ca?$	match a
ScT	?ca?$	S→T?
T?cT	?ca?$	T→ε
?cT	?ca?$	match ?
cT	ca?$	match c
T	a?$	T→aS
aS	a?$	match a
S	?$	S→T?
T?	?$	T→ε
?	?$	match ?
	$	accept

5. Consider the following grammar G over the alphabet $\{a, b, c\}$:

$$
\begin{aligned}
S' &\to S \\
S &\to Aa \\
S &\to Bb \\
A &\to Ac \\
A &\to \varepsilon \\
B &\to Bc \\
B &\to \varepsilon
\end{aligned}
$$

You want to implement G using an SLR(1) parser. Note that we have already added the $S' \to S$ production for you.

(a) Construct the first state of the LR(0) machine, compute the FOLLOW sets of A and B, and point out the conflicts that prevent the grammar from being SLR(1).

Solution:

5. (a) Prvo LR(0) stanje, FOLLOW skupovi i konflikti

Polazna gramatika:

$S' \to S$

$S \to Aa \mid Bb$

$A \to Ac \mid \varepsilon$

$B \to Bc \mid \varepsilon$

Prvo stanje LR(0) automata — zatvaranje skupa $\{S' \to \bullet S\}$:

I_0

$S' \to \bullet\, S$

$S \to \bullet\, A\, a$

$S \to \bullet\, B\, b$

$A \to \bullet\, A\, c$

$A \to \varepsilon \bullet$

$B \to \bullet\, B\, c$

$B \to \varepsilon \bullet$

Napomena: $A \to \varepsilon\bullet$ i $B \to \varepsilon\bullet$ su završene stavke (reduce akcije).

FOLLOW skupovi (za SLR analizu):

$\text{FOLLOW}(A) = \{\, a,\ c \,\}$ (iz $S \to Aa$ i $A \to Ac$)

$\text{FOLLOW}(B) = \{\, b,\ c \,\}$ (iz $S \to Bb$ i $B \to Bc$)

Konflikt koji sprečava SLR(1):

U stanju I_0 postoje dvije završene stavke: $A \to \varepsilon\bullet$ i $B \to \varepsilon\bullet$.

Terminal c je u oba FOLLOW skupa: $c \in \text{FOLLOW}(A)$ i $c \in \text{FOLLOW}(B)$.

Rezultat: reduce/reduce konflikt na terminalu c — parser ne zna da li da redukuje $A \to \varepsilon$ ili $B \to \varepsilon$, pa gramatika nije SLR(1).

(b) Show modifications to production 4 ($A \to Ac$) and production 6 ($B \to Bc$) to make the grammar SLR(1) while having the same language as the original grammar G. Explain the intuition behind this result.

Solution:

5. (b) Popravka gramatike

Modifikacija: zamijeni lijevu rekurziju desnom u produkcijama 4 i 6:

Produkcija 4 (stara): $A \to Ac$
Produkcija 4 (nova): $A \to cA$

Produkcija 6 (stara): $B \to Bc$
Produkcija 6 (nova): $B \to cB$

Popravljena gramatika (isti jezik):

$S \to Aa \mid Bb$
$A \to cA \mid \varepsilon$
$B \to cB \mid \varepsilon$

Novi FOLLOW skupovi:

$\text{FOLLOW}(A) = \{ a \}$ (samo iz $S \to Aa$; $A \to cA$ daje $\text{FOLLOW}(A)$ opet, ne širi)
$\text{FOLLOW}(B) = \{ b \}$ (samo iz $S \to Bb$)

Skupovi su sada disjunktni — nema reduce/reduce konflikta. Gramatika je SLR(1). ✓

Intuicija:

Uz lijevu rekurziju $A \to Ac$, terminal c se konzumira NAKON što je A već na steku. U početnom stanju parser vidi c u lookahead-u i ne zna da li gradi A (koje će završiti sa a) ili B (koje završava sa b).

Uz desnu rekurziju $A \to cA$, svaki c se odmah POMJERA (shift) na stek — nema potrebe za odlukom pri viđanju c. Razlika između A i B rješava se tek na kraju, kad parser vidi a (jedino u $\text{FOLLOW}(A)$) ili b (jedino u $\text{FOLLOW}(B)$) — a tada je odluka jednoznačna.